

CAPSTONE PROJECT

Enterprise Retail Analytics Platform on Azure

NAME: JAYAKUMAR T

BATCH: 7 (DATA ENGINEERING) ROLE BASED TRAINING

Github(Project Repository):

https://github.com/TKJayakumar/Retail_Capstone_DataEngineering

INDEX

S.I NO	CONTENT	PAGE NO
1	PROJECT OVERVIEW	3
2	BUSINESS SCENARIO	3
3	HIGH LEVEL ARCHITECTURE	3
4	HIGH LEVEL ARCHITECTURE OVERVIEW	4
5	LOW LEVEL ARCHITECTURE	4
6	LOW LEVEL ARCHITECTURE OVERVIEW	5
7	CONFIGURATIONS	6
7.1	CONFIGURATION VARIABLES	6
7.2	METADATA CONFIGURATIONS	7
8	MEDALLION ARCHITECTURE	
8.1	BRONZE LAYER	10
8.2	SILVER LAYER	11
8.3	SCD TYPE 2	13
8.4	GOLD LAYER	14
9	UNIT TESTING	18
10	AUDIT,LINEAGE AND MONITORING	18
11	TECHNOLOGY STACK	19
12	SCREEN SHOTS	20
13	CONCLUSION	30

Capstone Project

Enterprise Retail Analytics Platform on Azure

1. Project Overview

Students will design and implement an end-to-end Azure Data Engineering solution using Azure Data Factory (ADF), Azure Databricks (ADB), Azure Data Lake Storage Gen2, Azure SQL Database, Azure DevOps CI/CD, and Power BI. The solution must follow the Medallion Architecture (Bronze, Silver, Gold) and enterprise development practices.

2. Business Scenario

A retail company receives daily sales transactions from stores, customer master data from SQL Server, product master files, and exchange rates from a REST API. The business requires a centralized analytics platform for reporting and decision making.

3. ARCHITECTURE:

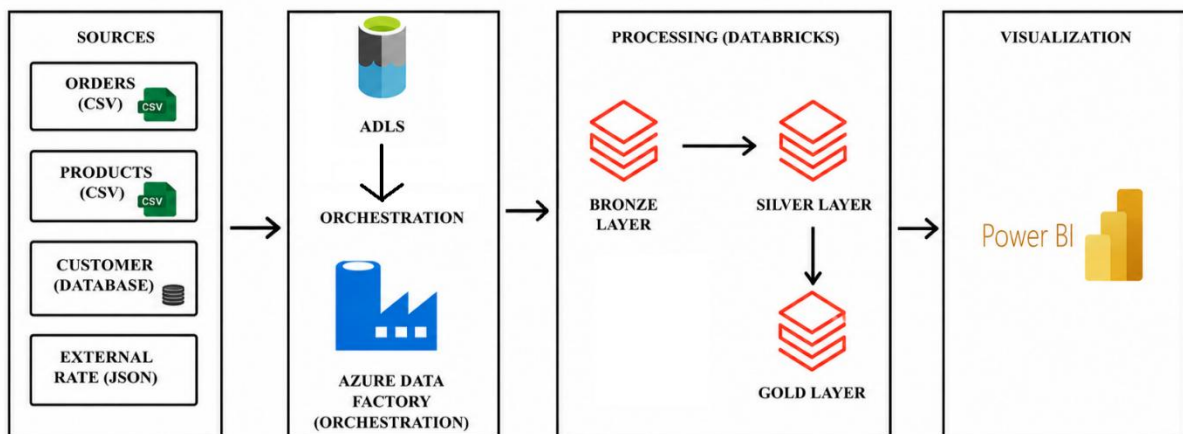


Fig 1 High Level Architecture Diagram

4. High Level ARCHITECTURE OVERVIEW:

Source Systems

1.Orders CSV Files

2.Products CSV Files

3.Customers Data

4.Exchange Rates JSON Files

Data Storage -> Azure Data Lake Storage Gen2

Processing Engine -> Azure Databricks

Data Layers

- Bronze Layer (Raw Data)
- Silver Layer (Validated & Cleansed Data)
- Gold Layer (Business Ready Data)

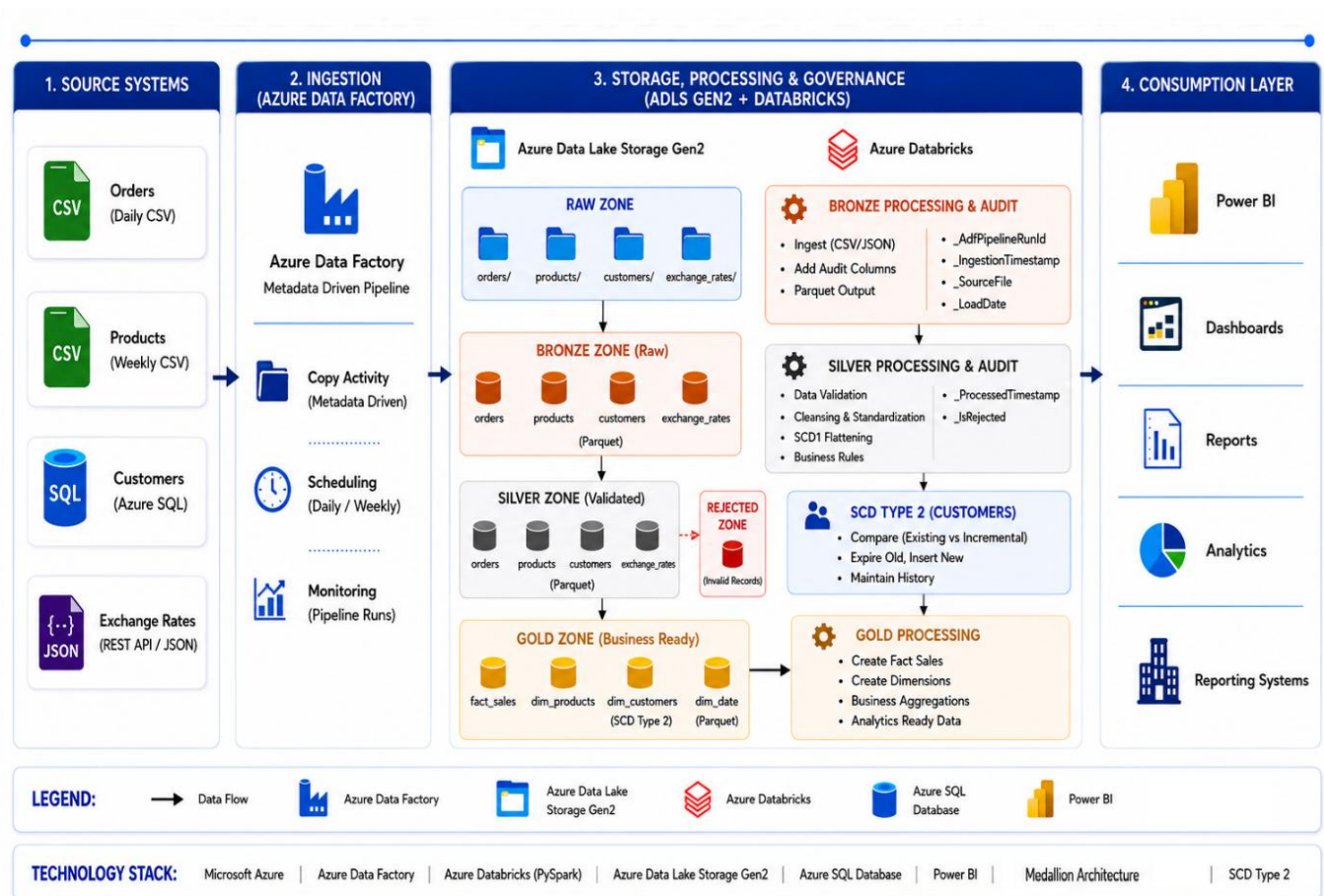
Consumption

1.Power BI

2.Analytics Dashboards

3.Reporting Systems

5.LOW LEVEL ARCHITECTURE:



6. Low Level Architecture overview

The Low Level Architecture diagram illustrates the end-to-end architecture of the Enterprise Retail Analytics Platform implemented on Microsoft Azure using the Medallion Architecture.

The process begins with multiple source systems, including Orders (CSV), Products (CSV), Customers (Azure SQL Database), and Exchange Rates (JSON/REST API). These datasets are ingested into the platform using Azure Data Factory (ADF) through a metadata-driven pipeline.

The ingested data is stored in Azure Data Lake Storage Gen2 (ADLS Gen2), where it is processed through the Bronze, Silver, and Gold layers using Azure Databricks

Bronze Layer: Ingests raw data, converts it into Delta format, and adds audit information while preserving the original source data.

Silver Layer: Performs data cleansing, validation, schema standardization, duplicate removal, and business rule enforcement. Invalid records are stored separately in the Rejected Zone.

SCD Type 2: Maintains historical customer information by tracking changes and preserving previous versions of customer records.

Gold Layer: Creates business-ready Fact and Dimension tables along with aggregated datasets that are optimized for reporting and analytics.

Finally, the Gold layer is consumed by Power BI, where interactive dashboards, reports, and analytical insights are generated to support business decision-making.

Technologies Used

- Microsoft Azure
- Azure Data Factory (ADF)
- Azure Data Lake Storage Gen2 (ADLS Gen2)
- Azure Databricks (PySpark)
- Delta Lake
- Azure SQL Database
- Power BI

Data Flow

Source Systems → Azure Data Lake Storage Gen2 → Azure Data Factory → Bronze Layer → Silver Layer → SCD Type 2 → Gold Layer → Power BI

7 Configurations

7.1 Configuration Variables

The Configuration File contains all the environment-specific settings required to run the Azure Data Engineering pipeline. These settings include the Azure Storage Account connection details, authentication credentials, and the storage paths used across the Bronze, Silver, SCD Type 2, and Gold layer notebooks. Instead of hardcoding these values throughout the notebooks, they are maintained as configuration variables. This makes the solution easier to maintain, simplifies deployment to different environments, and reduces the effort required when storage locations or credentials change.

Configuration Variables

Configuration Variable	Description	Purpose
STORAGE_KEY_CONFIG	Stores the Spark configuration property required to authenticate Azure Data Lake Storage Gen2.	Enables Spark to establish a secure connection with the Azure Storage Account before reading or writing data.
STORAGE_KEY_VALUE	Stores the access key of the Azure Storage Account.	Provides secure authentication between Azure Databricks and Azure Data Lake Storage Gen2.
METADATA_PATH	Specifies the location of the FileMetadata.csv file.	Allows the Bronze notebook to locate and read the metadata configuration file used for dynamic data ingestion.
RAW_PATH	Specifies the location of the raw source datasets.	Serves as the landing area where source files are stored before any processing takes place.
BRONZE_PATH	Specifies the storage location of the Bronze layer.	Stores the raw ingested datasets in Parquet format after adding audit information.
SILVER_PATH	Specifies the storage location of the Silver layer.	Stores cleansed, validated, and standardized datasets after all quality checks are completed.
REJECT_PATH	Specifies the storage location of the Rejected layer.	Stores records that fail validation rules so they can be reviewed and corrected later.
SCD_PATH	Specifies the storage location used for SCD Type 2 processing.	Stores historical and current customer records while maintaining version history.

Configuration Variable	Description	Purpose
GOLD_PATH	Specifies the storage location of the Gold layer.	Stores business-ready Fact tables, Dimension tables, and aggregated datasets used for reporting and analytics.

Working

Whenever a notebook starts executing, it first reads these configuration variables to establish a connection with Azure Data Lake Storage Gen2. The variables provide the required storage locations for reading input datasets and writing processed outputs. Since these settings are maintained separately from the processing logic, any changes to the storage account, authentication details, or folder structure can be handled by updating the configuration variables without modifying the notebook code.

7.2 Metadata Configuration

The project follows a **metadata-driven architecture**, where the data processing workflow is controlled using metadata files rather than hardcoded logic. Each metadata file is responsible for configuring a specific stage of the pipeline, such as data ingestion, schema definition, validation, duplicate handling, lookup validation, and Gold layer transformations. This design improves flexibility, scalability, and maintainability because new datasets or business rules can be introduced by updating the metadata files instead of changing the notebook code.

7.2.1 FileMetadata.csv

Purpose

FileMetadata.csv serves as the primary configuration file for the Bronze layer. It contains all the information required to identify the source datasets, locate them in Azure Data Lake Storage Gen2, determine their file formats, and specify where the processed output should be stored.

FileMetadata.csv Structure

Column Name	Description
FileName	Specifies the name of the source file that needs to be processed.
FileType	Defines the format of the source file, such as CSV or JSON.
RawPath	Specifies the location of the source file in Azure Data Lake Storage Gen2.
BronzePath	Specifies the destination path where the Bronze layer output will be stored.
Header	Indicates whether the source file contains column headers.

Working

During Bronze layer execution, the notebook first reads the **FileMetadata.csv** file. For every record in the metadata file, it retrieves the file name, file type, source path, destination path, and header information. Based on these values, the notebook dynamically reads the corresponding source file, adds audit columns, converts the data into Parquet format, and stores it in the Bronze layer. This approach allows new datasets to be processed simply by adding a new entry to the metadata file, without modifying the notebook.

7.2.2 SchemaMetadata.csv

Purpose

SchemaMetadata.csv defines the expected schema for every dataset processed by the pipeline. It ensures that each column is assigned the appropriate datatype before any validation or transformation takes place.

SchemaMetadata.csv Structure

Column Name	Description
DatasetName	Specifies the name of the dataset.
ColumnName	Specifies the name of the column.
DataType	Defines the expected datatype such as Integer, Double, String, or Date.
Nullable	Indicates whether NULL values are allowed for the column.

Working

Before processing a dataset, the Silver notebook reads **SchemaMetadata.csv** and applies the specified datatypes to each column. This ensures that all datasets follow a consistent schema and prevents errors caused by incorrect datatype inference during processing.

7.2.3 ValidationMetadata.csv

Purpose

ValidationMetadata.csv contains the business rules that are applied during Silver layer processing to ensure data quality.

ValidationMetadata.csv Structure

Column Name	Description
DatasetName	Specifies the dataset on which validation rules are applied.
ColumnName	Specifies the column to be validated.
ValidationRule	Defines the business validation rule, such as NOT NULL, GREATER THAN ZERO, or VALID EMAIL.
ErrorMessage	Stores the error message associated with the validation failure.

Working

While processing data in the Silver layer, the notebook reads the validation rules from **ValidationMetadata.csv** and applies them to each record. Records that satisfy all validation rules are stored in the Silver layer, whereas records that fail one or more validations are redirected to the Rejected layer for further analysis and correction.

7.2.4 DuplicateMetadata.csv

Purpose

DuplicateMetadata.csv defines the business key that should be used to identify duplicate records within each dataset.

DuplicateMetadata.csv Structure

Column Name	Description
DatasetName	Specifies the dataset where duplicate detection is performed.
PrimaryKey	Defines the business key or combination of columns used to identify duplicate records.

Working

During Silver layer processing, the notebook reads the duplicate configuration from **DuplicateMetadata.csv** and removes duplicate records based on the specified business key. This makes duplicate handling dynamic and eliminates the need to hardcode duplicate removal logic for every dataset.

7.2.5 LookupMetadata.csv

Purpose

LookupMetadata.csv defines the relationships between transactional datasets and their corresponding master datasets. It is used to perform referential integrity validation.

LookupMetadata.csv Structure

Column Name	Description
DatasetName	Specifies the source dataset containing the lookup field.
ColumnName	Specifies the foreign key column that requires validation.
LookupTable	Specifies the master dataset used for validation.
LookupColumn	Specifies the primary key column in the lookup dataset.

Working

During Silver layer processing, the notebook uses **LookupMetadata.csv** to validate foreign key relationships. For example, the **CustomerID** and **ProductID** values in the Orders dataset are verified against the corresponding Customers and Products datasets. Records containing invalid references are redirected to the Rejected layer to maintain data integrity.

7.2.6 GoldMetadata.csv

Purpose

GoldMetadata.csv controls the creation of business-ready analytical datasets in the Gold layer. It specifies the source datasets, target tables, and transformation type required to build reporting datasets.

GoldMetadata.csv Structure

Column Name	Description
TargetTable	Specifies the name of the Gold layer table to be created.

Column Name	Description
SourceTables	Lists the source datasets required to create the target table.
TransformationType	Defines the type of transformation, such as Dimension, Fact, or Aggregation.

Working

During Gold layer execution, the notebook reads **GoldMetadata.csv** to determine which datasets should be combined and what type of transformation should be applied. Based on this information, it creates Dimension tables, Fact tables, and aggregated reporting datasets before storing them in the Gold layer. New analytical tables can be introduced simply by updating this metadata file, without changing the notebook logic.

8. Medallion Architecture

8.1 Bronze Layer (Raw Ingestion)

The Bronze Layer serves as the centralized landing zone for all incoming source data.

Metadata Driven Ingestion

A metadata file (Filemetadata.csv) controls:

- File Name
- File Type
- Source Location
- Target Bronze Location
- Header Configuration

This enables reusable ingestion logic for multiple source files.

Data Loading

The Databricks ingestion framework dynamically reads:

- CSV Files
- JSON Files

based on metadata configurations.

Audit Columns Added

During ingestion, the following technical metadata fields are appended:

Column	Description
_AdfPipelineRunId	Tracks pipeline execution
_IngestionTimestamp	Records ingestion time
_SourceFile	Source file name
_LoadDate	Data load date

Storage Strategy

All raw datasets are stored in Azure Data Lake Storage Gen2 using Parquet format for efficient storage and retrieval.

Bronze Layer Objectives

- Preserve original source data
- Support data traceability
- Enable reprocessing if required
- Maintain historical raw records

8.2 Silver Layer (Validation and Conformance)

The **Silver Layer** transforms raw Bronze data into trusted, validated, and standardized datasets suitable for downstream processing. It performs data cleansing, schema standardization, duplicate removal, business rule validation, and data quality checks before loading the data into the Silver layer. Invalid records are redirected to the Rejected Layer for further analysis. The entire Silver layer is implemented using a **metadata-driven framework**, making it scalable and easy to maintain.

Data Cleansing Activities

The following operations are performed on all datasets during Silver layer processing.

Duplicate Removal

Duplicate records are removed based on the business key defined for each dataset.

Examples:

- **Products** – Duplicate records removed using **ProductID**
- **Customers** – Duplicate records removed using **CustomerID**
- **Orders** – Duplicate records removed using **OrderID**
- **Exchange Rates** – Duplicate records removed using **BaseCurrency**, **TargetCurrency**, and **RateDate**

Data Type Standardization

Business columns are converted into appropriate data types.

Examples:

- ProductID → Integer
- CustomerID → Integer
- OrderID → Integer
- Quantity → Integer
- CostPrice → Double
- UnitPrice → Double
- ExchangeRate → Double

Dataset-Level Validation

Products

The Products dataset undergoes the following validations:

- ProductID cannot be NULL.
- ProductName cannot be empty.
- Category must contain a valid value.
- Brand cannot be **UnknownBrand**.
- CostPrice must be greater than zero.

Customers

The Customers dataset undergoes the following validations:

- CustomerID cannot be NULL.
- First Name and Last Name cannot be empty.
- Email must follow a valid email format.
- Phone number must contain exactly 10 digits.
- City and State cannot be NULL.
- LastUpdated must contain a valid date.

Orders

The Orders dataset undergoes the following validations:

- OrderID cannot be NULL.
- CustomerID cannot be NULL.
- ProductID cannot be NULL.
- OrderDate must contain a valid date.
- Quantity must be greater than zero.
- UnitPrice must be greater than zero.
- StoreCode cannot be empty.
- CustomerID must exist in the Customers dataset.
- ProductID must exist in the Products dataset.

Exchange Rates

The Exchange Rates dataset undergoes the following validations:

- BaseCurrency cannot be NULL.
- TargetCurrency cannot be NULL.
- ExchangeRate must be greater than zero.
- RateDate must contain a valid date.
- Only supported currencies (EUR, GBP, INR, JPY, AUD, CAD) are accepted.

Additionally, JSON exchange rate data is parsed and flattened into relational columns for analytical processing.

Data Quality Routing

Records that fail validation rules are not discarded.

Valid Records

- Stored in the **Silver Layer**
- Used for SCD Type 2 processing and Gold layer transformations

Invalid Records

- Stored in the **Rejected Layer**
- Preserved for data quality analysis and troubleshooting

Audit Columns Added

Column	Description
<code>_ProcessedTimestamp</code>	Records the timestamp of Silver layer processing
<code>_IsRejected</code>	Indicates whether the record is valid or rejected

Metadata-Driven Framework

The Silver layer is fully metadata-driven. Validation rules, schema definitions, duplicate checks, and lookup validations are maintained in metadata files instead of hardcoded in the notebooks.

Metadata files used:

- FileMetadata.csv
- SchemaMetadata.csv
- ValidationMetadata.csv
- DuplicateMetadata.csv
- LookupMetadata.csv

This approach allows new datasets and validation rules to be added with minimal code changes, improving scalability, maintainability, and reusability.

8.3 Slowly Changing Dimensions (SCD Type 2)

Customer data is managed using SCD Type 2 methodology to preserve historical changes.

Business Requirement

When customer information changes:

- City
- State
- Email
- Phone

the previous version must be retained.

Incremental Processing

Customer records are updated and stored in:

`customers_incremental.parquet`

Change Detection

The framework compares:

- Existing Customer Records
- Incremental Customer Records

to identify:

- Changed Records
- Unchanged Records

Historical Tracking Columns

Column	Description
CustomerKey	Surrogate Key
EffectiveDate	Version Start Date
ExpiryDate	Version End Date
IsCurrent	Current Active Record

SCD Workflow

Old Version

- Marked as inactive
- Expiry Date populated

New Version

- New surrogate key generated
- Marked as current

Unchanged Records

- Retained as active records

Benefits

- Historical customer analysis
- Customer movement tracking
- Point-in-time reporting

8.4 Gold Layer (Business Intelligence Layer)

The **Gold Layer** is the final business-ready layer of the Medallion Architecture. It transforms validated Silver data into analytical datasets optimized for reporting and business intelligence. The Gold layer creates Dimension tables, Fact tables, and aggregated datasets that are directly consumed by Power BI dashboards and business users. The Gold layer is implemented using a **metadata-driven framework**, allowing new analytical tables to be added with minimal code changes.

Business Transformation Activities

The following transformations are performed while creating the Gold layer.

Dimension Table Creation

Dimension tables are created from validated Silver and SCD datasets.

DimProducts

- Created from Silver Products.
- Contains ProductID, ProductName, Category, SubCategory, Brand, and CostPrice.
- Used as the Product Dimension for reporting.

DimCustomers

- Created from the SCD Type 2 Customer table.

- Includes only current customer records (IsCurrent = True).
 - Preserves CustomerKey, EffectiveDate, ExpiryDate, and customer attributes.
-

Fact Table Creation

The **FactSales** table is created by joining multiple datasets.

Data sources:

- Silver Orders
- Gold DimProducts
- Gold DimCustomers
- Silver Exchange Rates

Business calculations performed:

- $\text{SalesAmount} = \text{Quantity} \times \text{UnitPrice}$
- $\text{CostAmount} = \text{Quantity} \times \text{CostPrice}$
- $\text{Profit} = \text{SalesAmount} - \text{CostAmount}$
- Loss calculation for negative margins
- Exchange Rate mapping
- Order Year
- Order Month
- Month Name
- Quarter
- Day of Week

The FactSales table serves as the central fact table for analytical reporting.

Business Aggregation

The Gold layer creates multiple analytical summary tables.

Sales Summary

Generates business KPIs grouped by:

- Year
- Month
- Category
- Store

Metrics calculated:

- Total Sales
- Total Cost
- Total Profit

- Total Quantity
 - Total Orders
 - Average Order Value
 - Average Unit Price
-

Store Sales

Generates store-level performance metrics.

Metrics include:

- Total Orders
 - Total Customers
 - Total Products
 - Total Sales
 - Total Cost
 - Total Profit
 - Profit Margin
 - Store Performance Rating
-

Monthly Sales

Generates monthly sales trends.

Metrics include:

- Monthly Sales
 - Monthly Profit
 - Previous Month Sales
 - Sales Growth Percentage
 - Monthly Performance Status
-

Business Rules Applied

The following business rules are implemented:

- Current customer records are selected using **IsCurrent = True**.
- Sales amount is calculated from Quantity and Unit Price.
- Cost amount is calculated using Product Cost Price.
- Profit and Loss are derived from Sales and Cost.
- Exchange rates are joined to support currency conversion.
- Time-based attributes are generated for trend analysis.
- Summary tables are created using aggregation functions.

Gold Layer Outputs

The Gold layer produces the following analytical datasets:

- DimProducts
- DimCustomers
- FactSales
- SalesSummary
- StoreSales
- MonthlySales

These datasets are optimized for reporting, dashboard creation, and business analytics.

Metadata-Driven Framework

The Gold layer is implemented using a metadata-driven framework.

Metadata file used:

- GoldMetadata.csv

The framework automatically:

- Reads source datasets.
- Identifies the transformation type.
- Creates Dimension tables.
- Creates Fact tables.
- Generates analytical summary tables.
- Writes the results to the Gold layer.

This approach improves scalability, maintainability, and allows new business datasets to be added with minimal development effort.

Gold Layer Objectives

- Create business-ready analytical datasets.
- Build Fact and Dimension tables.
- Generate KPIs and aggregated reports.
- Support historical customer analytics using SCD Type 2.
- Improve reporting performance.
- Enable interactive Power BI dashboards.
- Deliver trusted and optimized data for business decision-making.

9.UNIT TESTING:

Unit testing was performed to validate each stage of the data engineering pipeline and ensure that the individual components functioned correctly before end-to-end execution. The testing focused on data ingestion, transformation, validation, and output generation across the Bronze, Silver, SCD Type 2, and Gold layers.

Test Case	Description	Expected Result	Status
Source File Validation	Verify that source files are read successfully from the configured location.	Files are loaded without errors.	Passed
Metadata Validation	Verify that metadata files are read correctly.	Metadata configurations are loaded successfully.	Passed
Bronze Layer Testing	Validate ingestion of raw data with audit columns.	Data is stored in the Bronze layer with audit information.	Passed
Schema Validation	Verify that the configured schema is applied correctly.	All columns have the expected data types.	Passed
Data Validation	Validate business rules such as NOT NULL, valid email, and positive values.	Valid records move to Silver and invalid records move to the Rejected layer.	Passed
Duplicate Validation	Verify duplicate removal based on business keys.	Duplicate records are removed successfully.	Passed
Lookup Validation	Validate CustomerID and ProductID against master datasets.	Invalid lookup records are redirected to the Rejected layer.	Passed
SCD Type 2 Testing	Verify historical tracking of customer records.	New versions are created, and previous records are expired correctly.	Passed
Gold Layer Testing	Validate creation of Fact, Dimension, and Aggregation tables.	Business-ready datasets are generated successfully.	Passed
Pipeline Execution	Verify end-to-end execution of the ADF Master Pipeline.	Pipeline completes successfully without failures.	Passed
Power BI Validation	Verify data availability in Power BI reports.	Reports display the latest processed data correctly.	Passed

10. Audit, Lineage, and Monitoring

To ensure enterprise-grade governance and observability, audit metadata is maintained throughout the pipeline.

Bronze Layer Tracking

Column

_AdfPipelineRunId

_IngestionTimestamp

Column

_SourceFile

_LoadDate

Silver Layer Tracking

Column

_ProcessedTimestamp

_IsRejected

Benefits

- End-to-end data lineage
- Data traceability
- Regulatory compliance
- Root cause analysis
- Pipeline monitoring

11. Technology Stack

Cloud Platform

Microsoft AzureStorage->Azure Data Lake Storage Gen2

Processing Engine -> Azure Databricks

Programming Language ->PySpark

Architecture Pattern ->Medallion Architecture

Data Modeling -> Star Schema & SCD Type 2

Reporting ->Power BI

12.SCREEN SHOT:

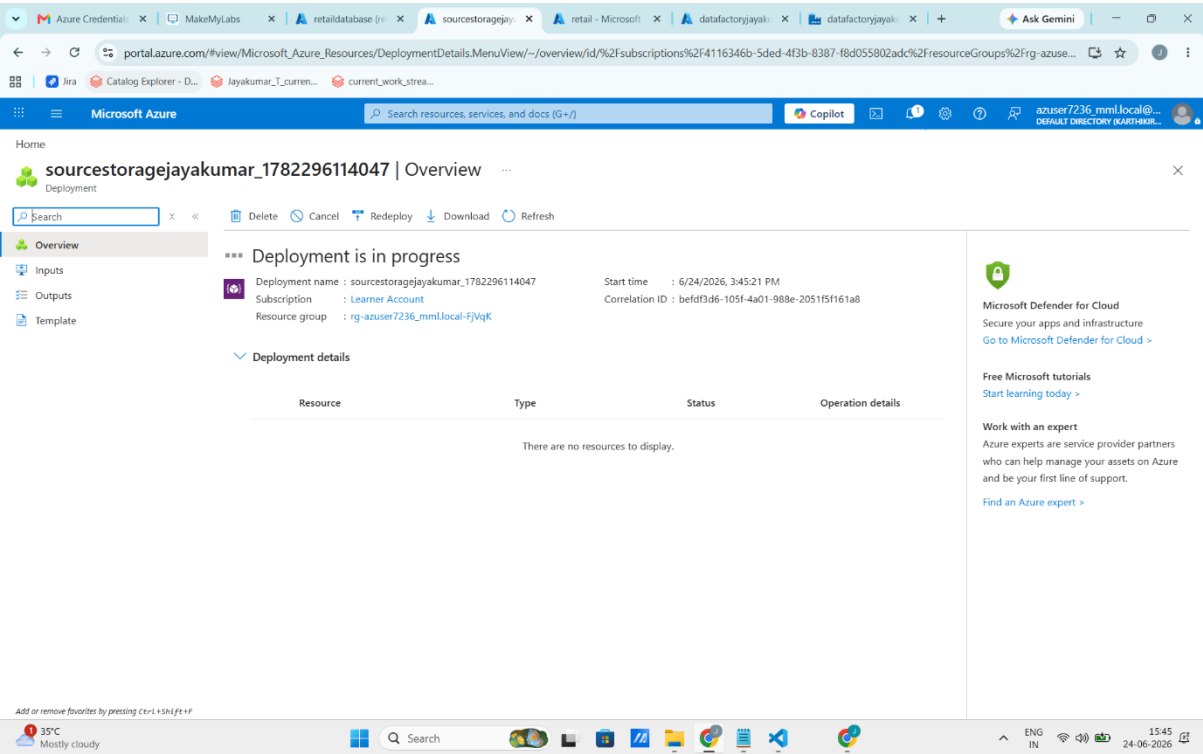


FIG 1 :SOURCE STORAGE ACCOUNT

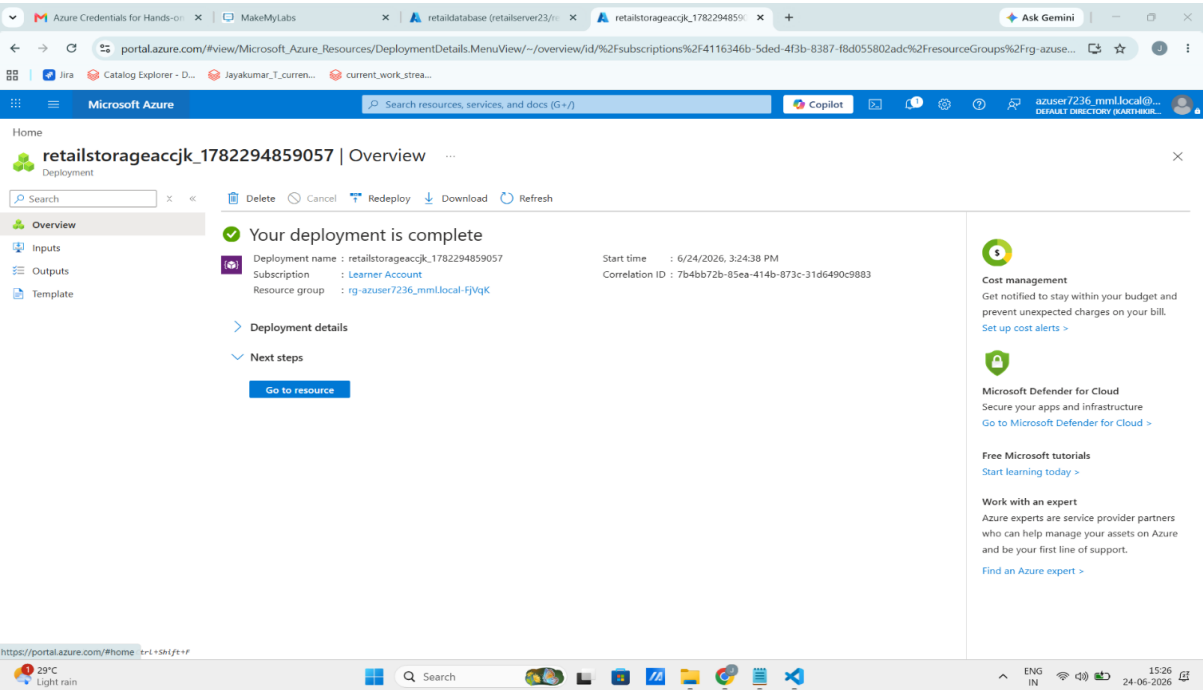


FIG 2: ADLS STORAGE ACCOUNT

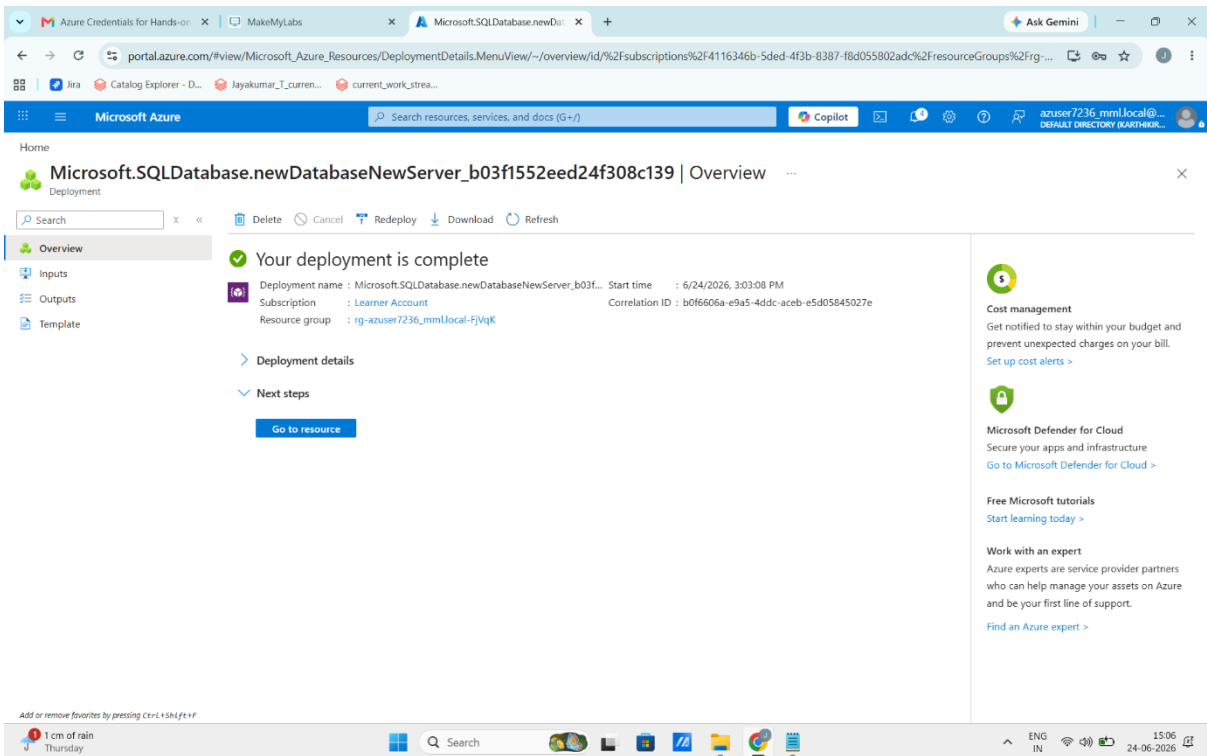


FIG 3: SQL SERVER STORAGE CREATION

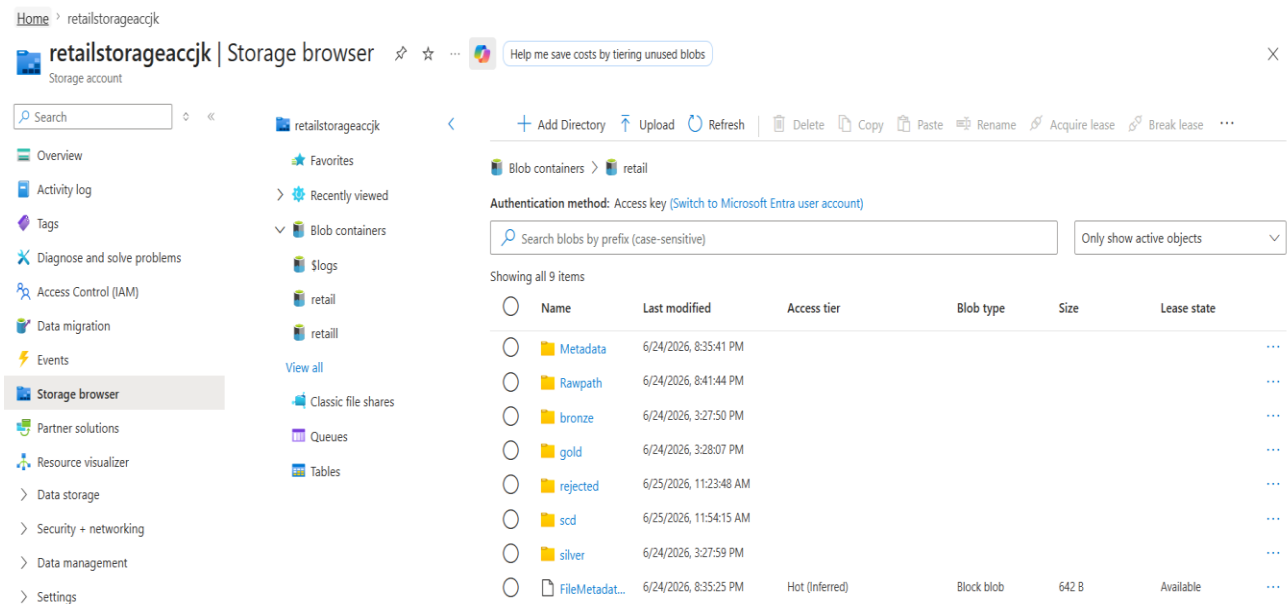


FIG 4 CONTAINERS

Microsoft Azure portal showing the Query editor (preview) for a retail database. The interface displays a table of customer data with columns: CustomerID, First Name, Last Name, Email, Phone, City, State, and LastUpdated. The table shows 18 rows of data.

CustomerID	First Name	Last Name	Email	Phone	City	State	LastUpdated
1	NULL	White	david.white1@gmail...	9440030663	Austin	IL	2025-02-16
2	Emma	Thomas	emma.thomas2@gmail...	9917489574	Boston	IL	NULL
3	Robert	Clark	NULL	9650229169	Phoenix	IL	2025-01-14
4	Jane	Lewis	jane.lewis4@gmail.co...	9742144634	Austin	CO	2025-01-07
5	James	Johnson	james.johnson5@gmail...	9878295151	Atlanta	WA	2025-10-20
6	James	Brown	james.brown6@gmail...	9249078381	Boston	MA	2025-05-18
7	Jane	Brown	jane.brown7@gmail...	9566730092	Boston	CO	2025-06-03
8	Jane	Thomas	jane.thomas8@gmail...	9370260533	Seattle	GA	2025-11-01
9	Emma	Johnson	emma.johnson9@gmail...	9245480775	Boston	NY	2025-07-22
10	Sophia	White	sophia.white10@gmail...	9963370520	Austin	GA	2025-06-21
11	Sophia	Brown	sophia.brown11@gmail...	942589929	Austin	WA	2025-09-03
12	James	Thomas	james.thomas12@gmail...	9412940071	Denver	WA	2025-06-16
13	James	Thomas	james.thomas13@gmail...	9222286118	Miami	WA	2025-10-31
14	David	Clark	david.clark14@gmail...	9128900946	Denver	WA	2025-08-13
15	Jane	White	jane.white15@gmail...	9822604249	Denver	TX	2025-11-21
16	Michael	Brown	michael.brown16@gmail...	9867979226	Phoenix	CO	2025-02-07
17	Sophia	Martin	sophia.martin17@gmail...	9725179752	Phoenix	FL	2025-09-17
18	James	Clark	james.clark18@gmail...	9139476288	Austin	FL	2025-11-01

FIG 5 DATABASE

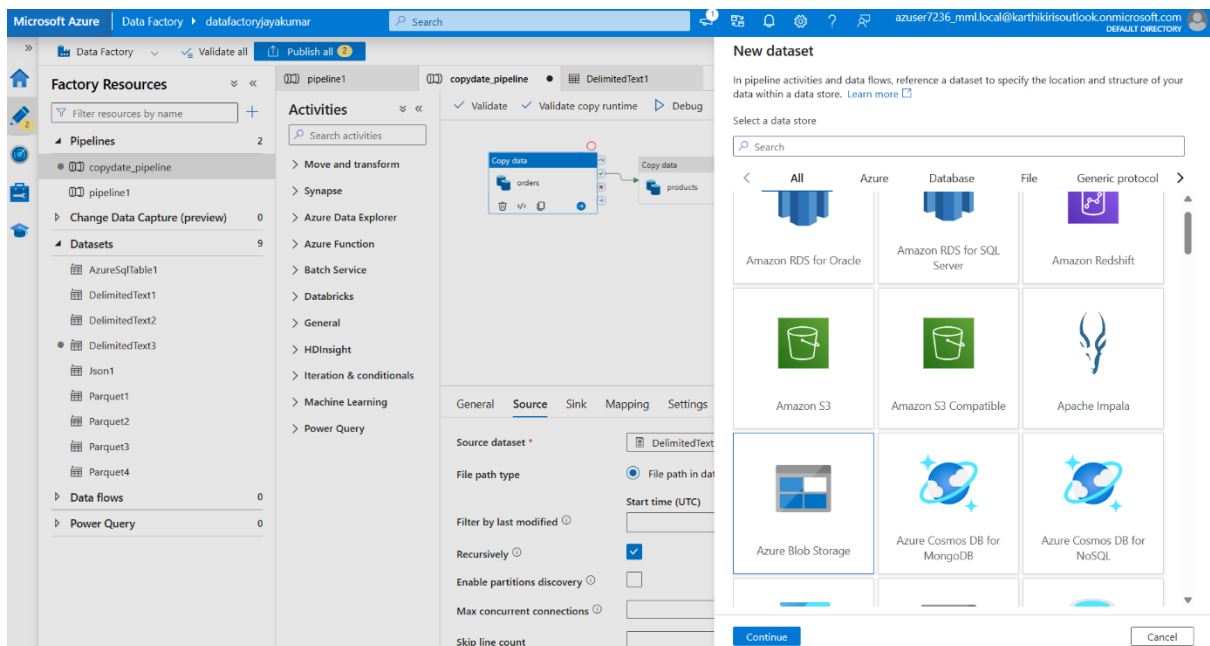


FIG 6 COPY DATA FROM SOURCE TO ADLS

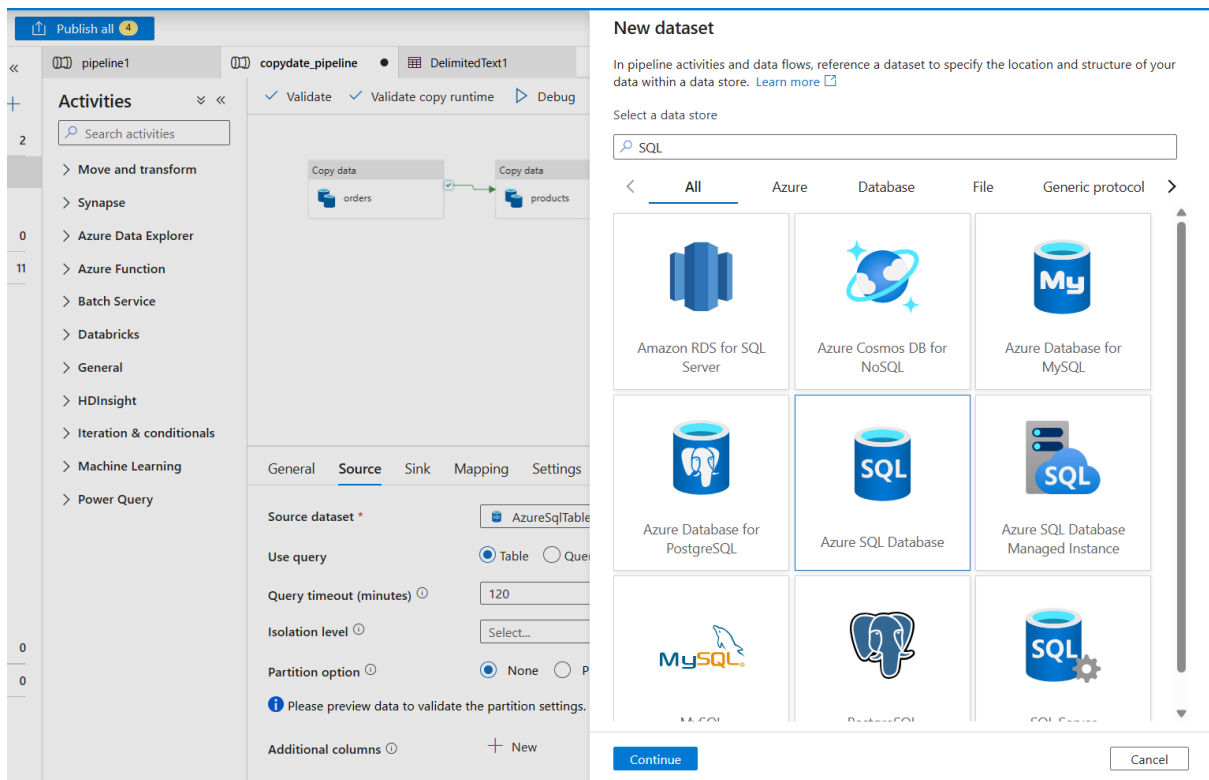


FIG 7 AZURE SQL

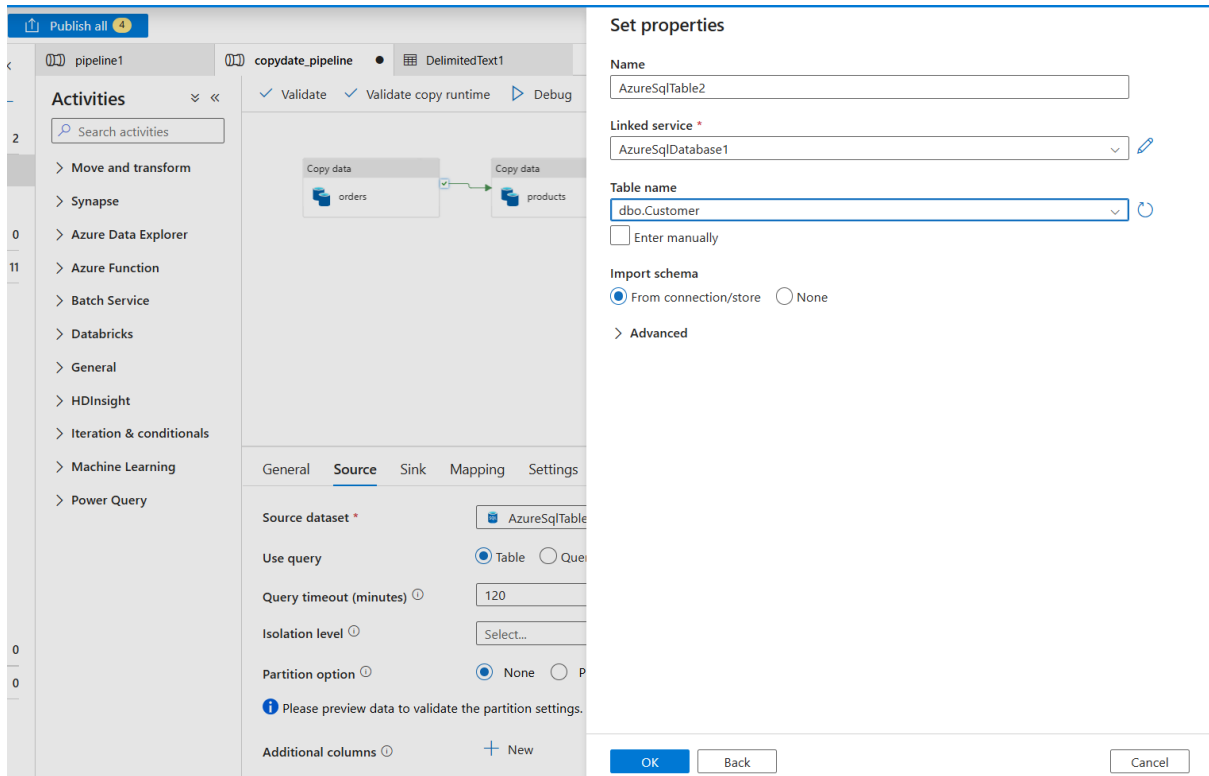


FIG 8 AZURE SQL TABLE

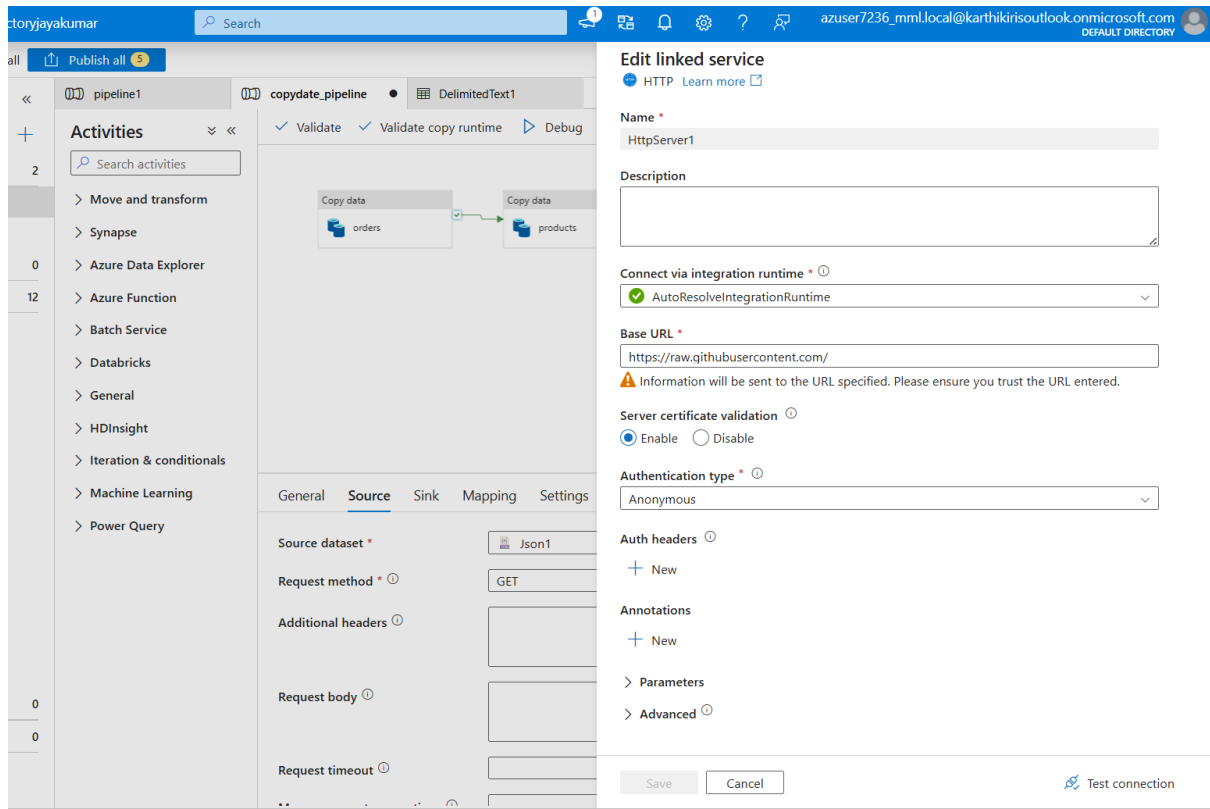


FIG 9 DATA FROM HTTP FORMAT

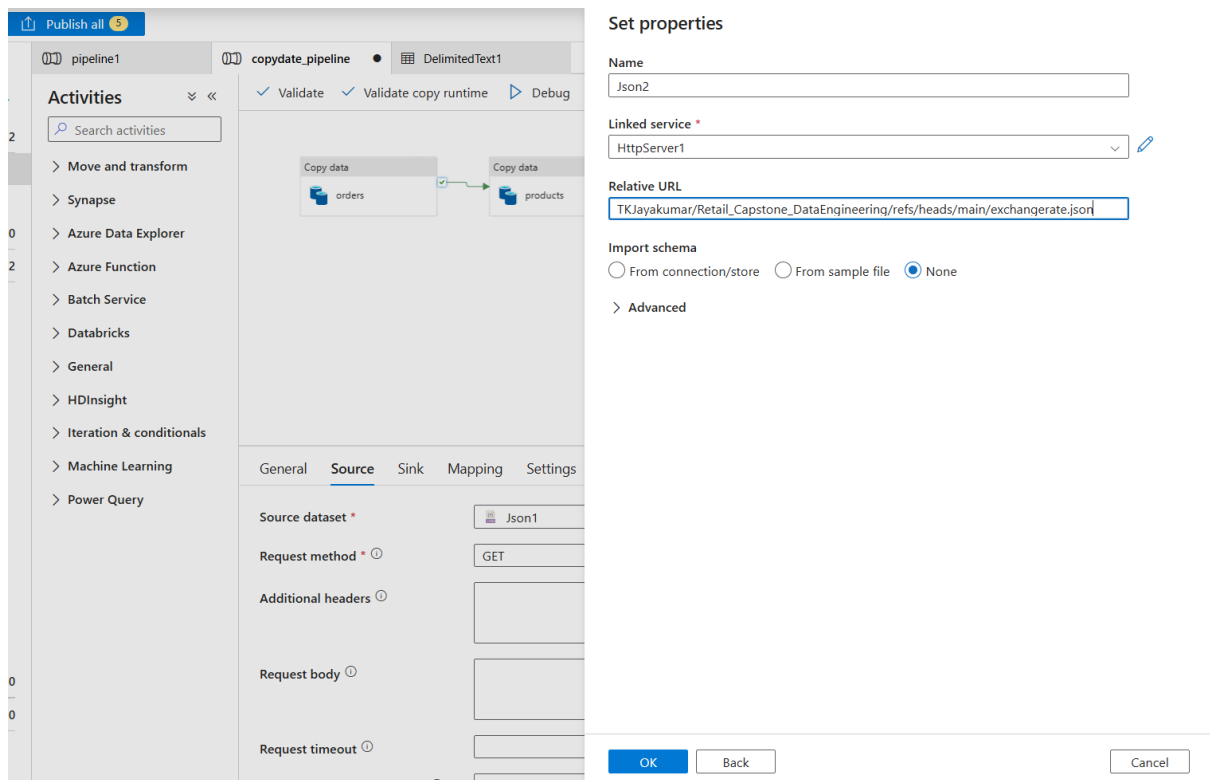


FIG 10 HTTP FOMRAT RELATIVE URL

Factory Resources

- Pipelines: 1
 - copydate_pipeline
- Datasets: 8
- Data flows: 0
- Power Query: 0

Activities

- Copy data

Copy data pipeline diagram:

```

graph LR
    A[Copy data: orders] --> B[Copy data: products]
    B --> C[Copy data: customers]
    C --> D[Copy data: exchangerate]
  
```

Output

Activity name	Activity status	Activity name	Run start	Duration	Integration runtime
exchangerate	Succeeded	Copy data	6/26/2026, 10:02:13 AM	18s	AutoResolveIntegrationRuntime (East US)
customers	Succeeded	Copy data	6/26/2026, 10:01:52 AM	20s	AutoResolveIntegrationRuntime (East US)
products	Succeeded	Copy data	6/26/2026, 10:01:27 AM	24s	AutoResolveIntegrationRuntime (East US)
orders	Succeeded	Copy data	6/26/2026, 10:00:59 AM	27s	AutoResolveIntegrationRuntime (East US)

FIG 11.1 COPY DATA

Activities

- Move and transform
- Synapse
- Azure Data Explorer
- Azure Function
- Batch Service
- Databricks
- General
- HDI Insight
- Iteration & conditionals
- Machine Learning
- Power Query

Lookup activity: LookupCustomerData

ForEach loop: ForEachCustomerRow

Stored procedure activity: Stored procedure...

Output

Activity name	Activity status	Activity name	Run start	Duration	Integration runtime
Stored procedure2	Succeeded	Stored procedu	6/29/2026, 1:23:14 PM	8s	AutoResolveIntegrationRuntime (East US)
Stored procedure2	Succeeded	Stored procedu	6/29/2026, 1:23:06 PM	6s	AutoResolveIntegrationRuntime (East US)
Stored procedure2	Succeeded	Stored procedu	6/29/2026, 1:23:05 PM	6s	AutoResolveIntegrationRuntime (East US)
Stored procedure2	Succeeded	Stored procedu	6/29/2026, 1:22:59 PM	6s	AutoResolveIntegrationRuntime (East US)

FIG 11.2 LOOKUP AND FOR EACH ACTIVITY

The screenshot shows the Azure Data Factory interface for a pipeline named 'copydate_pipeline'. The pipeline consists of three Notebook activities: 'raw_to_bronze', 'bronze_to_silver', and 'silver_to_gold', all of which are marked as 'Succeeded'. The interface includes a left sidebar with 'Activities' and 'Synapse' sections, and a bottom section showing the 'Output' tab with a table of activity details.

Activity name	Activity status	Activity type	Run start	Duration	Integration runtime
silver_to_gold	Succeeded	Notebook	6/26/2026, 2:23:35 PM	53s	AutoResolveIntegrationRuntime (East US)
bronze_to_silver	Succeeded	Notebook	6/26/2026, 2:22:11 PM	1m 24s	AutoResolveIntegrationRuntime (East US)
raw_to_bronze	Succeeded	Notebook	6/26/2026, 2:21:17 PM	53s	AutoResolveIntegrationRuntime (East US)

FIG 11.3 COPY DATA RAW TO GOLD LAYER USING PIPELINE

The screenshot shows the Azure Data Factory interface for a pipeline named 'MASTER_PIPELINE'. The pipeline consists of a sequence of Copy data activities: 'orders', 'products', 'customers', and 'exchangerate', followed by three Notebook activities: 'raw_to_bronze', 'bronze_to_silver', and 'silver_to_gold'. All activities are marked as 'Succeeded'. The interface includes a left sidebar with 'Factory Resources' and 'Activities' sections, and a bottom section showing the 'Output' tab with a table of activity details.

Activity name	Activity status	Activity type	Run start	Duration	Integration runtime
silver_to_gold	Succeeded	Notebook	6/26/2026, 5:02:52 PM	1m 10s	AutoResolveIntegrationRuntime (East US)
bronze_to_silver	Succeeded	Notebook	6/26/2026, 5:00:51 PM	1m 59s	AutoResolveIntegrationRuntime (East US)
raw_to_bronze	Succeeded	Notebook	6/26/2026, 4:59:08 PM	1m 42s	AutoResolveIntegrationRuntime (East US)
exchangerate	Succeeded	Copy data	6/26/2026, 4:58:48 PM	18s	AutoResolveIntegrationRuntime (East US)
customers	Succeeded	Copy data	6/26/2026, 4:58:25 PM	22s	AutoResolveIntegrationRuntime (East US)
products	Succeeded	Copy data	6/26/2026, 4:57:59 PM	26s	AutoResolveIntegrationRuntime (East US)
orders	Succeeded	Copy data	6/26/2026, 4:57:35 PM	23s	AutoResolveIntegrationRuntime (East US)

FIG 11.4 MASTER PIPELINE



FIG 11.5 POWER BI

Home > retaildatabase (retailserver23/retaildatabase)

retaildatabase (retailserver23/retaildatabase) | Query editor (preview) ☆ ...

SQL database

Search

Overview

Activity log

Tags

Diagnose and solve problems

Query editor (preview)

Mirror database in Fabric (preview)

Resource visualizer

Settings

Compute + storage

Connection strings

Maintenance

Properties

Explorer

Search

dbo

Tables

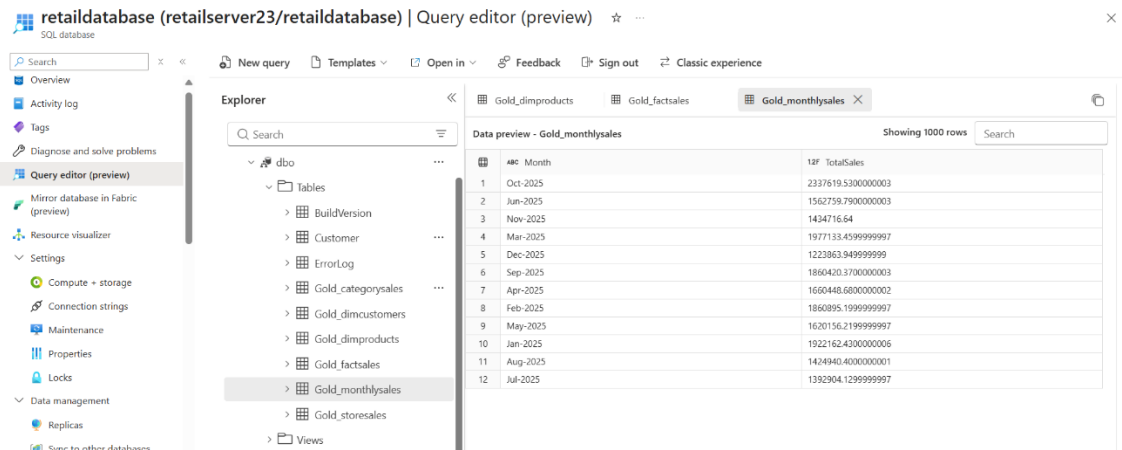
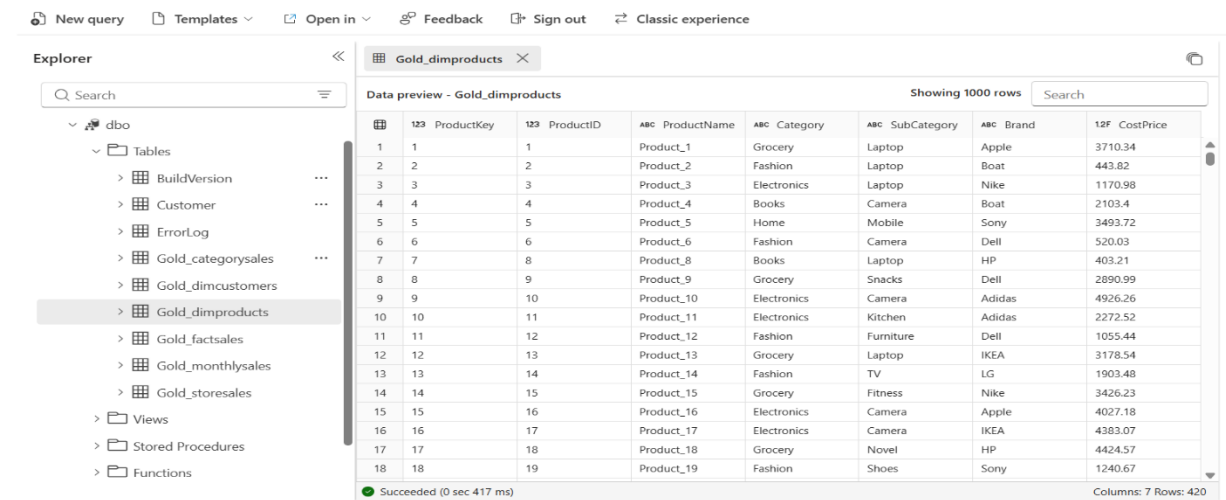
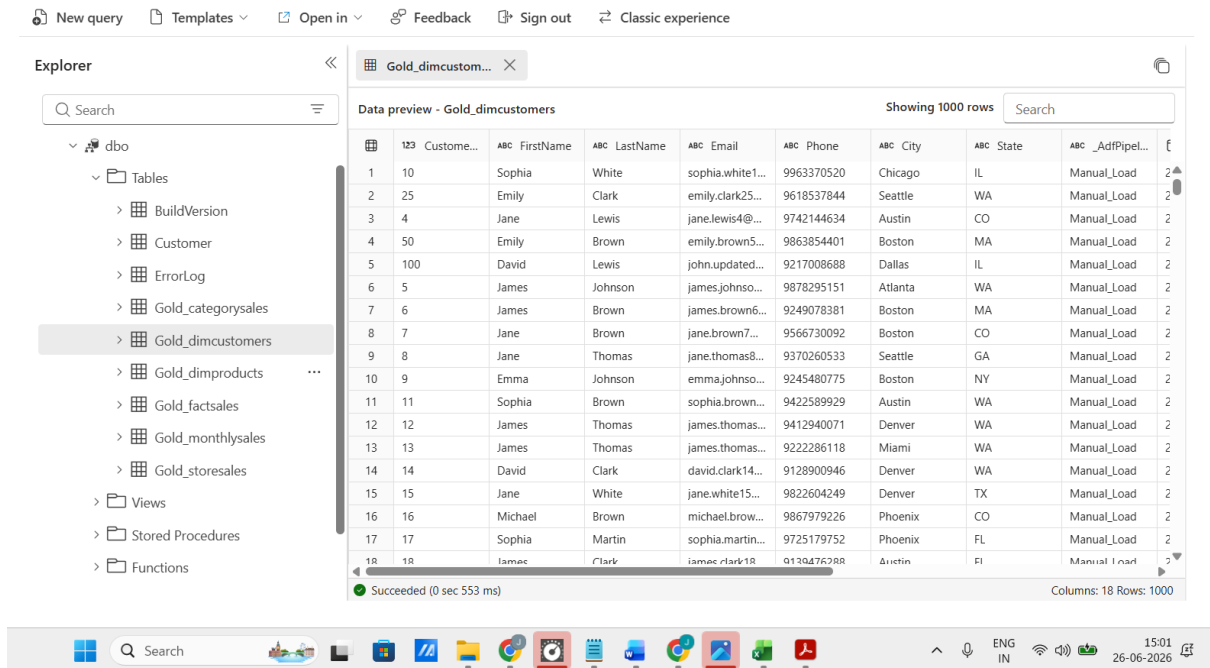
- BuildVersion
- Customer
- ErrorLog
- Gold_categorysales
- Gold_dimcustomers
- Gold_dimproducts
- Gold_factsales

Data preview - Gold_categorysales

Showing 1000 rows

	ABC Category	12P TotalSales	12L TotalQuantity
1	Home	3864033.850000003	1474
2	Fashion	2063282.9800000004	903
3	Sports	2962110.190000001	1140
4	Grocery	4232699.19	1648
5	Electronics	3291710.1800000025	1359
6	Books	3864184.410000002	1529

FIG 12.1 GOLD LAYER OUTPUT 1



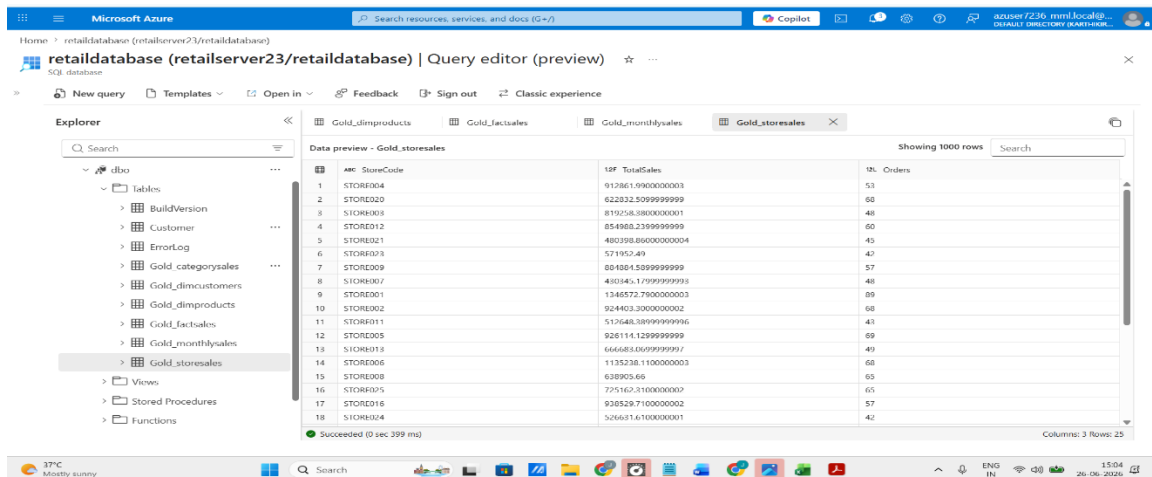


FIG 12.5 GOLD LAYER OUTPUT 5

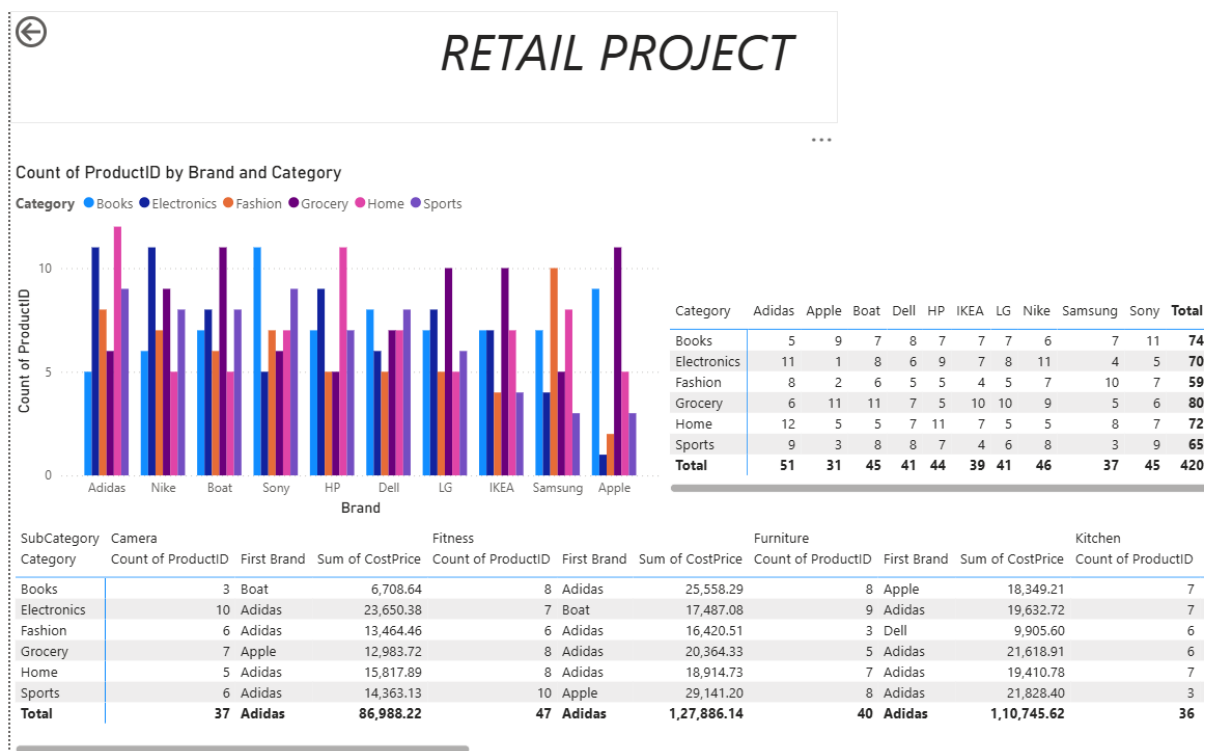


FIG 13.1 POWER BI OUTPUT

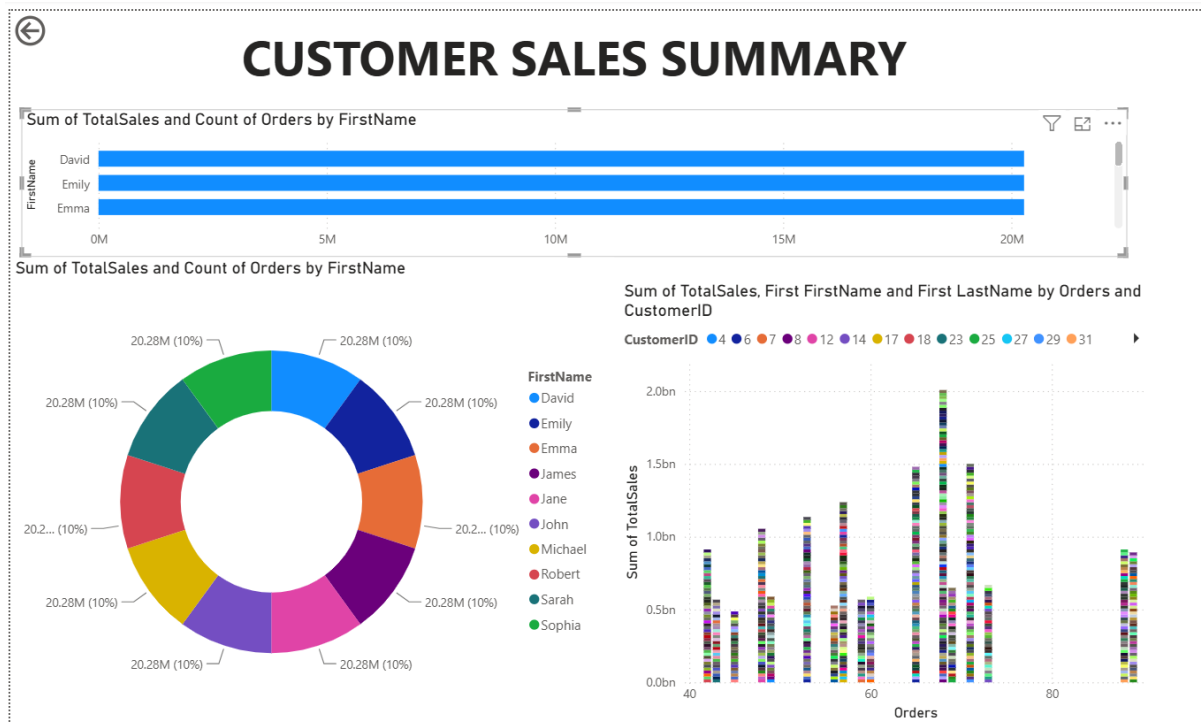


FIG 13.2 POWER BI OUTPUT

12.CONCLUSION:

- **Centralized Data Integration:** Successfully ingests data from CSV files (Orders, Products), Azure SQL Database (Customers), and REST API (Exchange Rates) into Azure Data Lake Storage Gen2.
- **Metadata-Driven Pipeline:** Uses configuration and metadata files to dynamically control ingestion, schema validation, duplicate removal, lookup validation, and Gold layer transformations, minimizing code changes when onboarding new datasets.
- **Medallion Architecture Implementation:**
 - Bronze Layer: Stores raw data in Delta/Parquet format with audit metadata while preserving source data.
 - Silver Layer: Cleanses, validates, standardizes, removes duplicates, and separates invalid records into a Rejected Zone.
 - Gold Layer: Produces business-ready Fact and Dimension tables along with aggregated datasets optimized for reporting.
- **Historical Data Management:** Implements Slowly Changing Dimension (SCD Type 2) to preserve customer history and enable point-in-time analysis.

- **Business Intelligence:** Generates analytical datasets such as FactSales, DimCustomers, DimProducts, SalesSummary, StoreSales, and MonthlySales, enabling interactive Power BI dashboards and KPI reporting.
- **Data Governance:** Maintains audit columns, lineage, and monitoring information throughout the pipeline to support traceability, compliance, and troubleshooting.
- **Enterprise Best Practices:** Leverages Azure Data Factory, Azure Databricks (PySpark), Azure SQL Database, Delta Lake, ADLS Gen2, and Power BI following scalable enterprise development practices.